

Iteratorer

Iteratorer är den nästvikigaste gruppen av klasser i STL. Via iteratorer ges programmeraren åtkomst till alla element i en container. Man kan se på en iterator som en "pekare" in i en container som pekare på ett element. Via denna pekare kan man sedan accessera elementet i just denna position. Multipla iteratorer kan användas på samma container, de kan t.o.m. peka på samma element. Iteratorer är enkla klasser men de innehåller funktionalitet för att gå till nästa eller föregående element i containern, för att på så vis ge möjligheter att enkelt iterera igenom alla element. Alla containers stöder iteratorer.

Det finns några olika typer av iteratorer som fungerar på olika sätt. Alla iteratorer stöds inte av alla containers beroende på det sätt containern är implementerad. De olika typerna är:

- *framåt-iteratorer* kan endast iterera åt ett håll. Från det ställe där de startar kan de endast gå mot den andra ändan av containern. T.ex. en lista itereras enklast igenom med en dylik iterator. Man kan gå antingen framåt eller bakåt, men inte åt båda hållen.
- *framåt och bakåtgående iteratorer* kan iterera framåt och bakåt i den container de opererar på. Alla containers kan inte använda denna typ av iteratorer.
- *random access-iteratorer* som kan accessera en container på vilket sätt som helst, samt hoppa från element till ett annat utan att de måste vara åtföljande element.

Använda iteratorer

Ett exempel på hur man kan iterera igenom en container ges nedan:

```
deque<float> Scores;
...
deque<float>::iterator Start = Scores.begin ();
deque<float>::iterator End = Scores.end ();
for ( ; Start != End; Start++ ) {
    // skriv ut värdet
    cout << *Start;
}
```

Varje container har metoderna `start()` och `end()` för att returnera iteratorer som pekar på det första respektive förbi det sista elementet. Men förbi det sista elementet avses en iterator som itererat ända till slutet och sedan "tar ett steg" till. Denna iterator innehåller ett illegalt värde och kan användas för att kontrollera när man itererat genom hela containern. Många metoder returnerar `end()` för att indikera att någonting inte kunde utföras eller hittas. Metoden `find()` returnerar `end()` om det sökta elementet inte kunde hittas.

Många av de olika metoderna för containers använder sig av iteratorer för att markera vilket element som skall accesseras. T.ex. metoderna `erase()` och `insert()` använder sig av iteratorer för att markera det element som skall raderas respektive den plats där ett nytt element skall placeras in. Metoden `erase()` finns vanligen i en version som tar två iteratorer som parametrar. Dessa markerar då det första och det sista elementet i en mängd element som skall raderas.

[Förutgående](#)
Containers

[Hem](#)
[Upp](#)

[Nästa](#)
Generiska algoritmer